GNSS-Disciplined Cuckoo Clock:

An Interdisciplinary Senior Design Project

Nathaniel Frissell, W2NAF, University of Scranton • October 28, 2026



The finished clock at home: the Shortt-style master in its vacuum tank driving the cuckoo's slave pendulum, Nixie and dial agreeing on the time, the Teletype console, the card reader, and the GNSS antenna and WWV receiver. *(Illustration.)*

Project overview

Students will build a mechanical cuckoo clock whose timekeeping is disciplined to GNSS, running control firmware written in the Shakespeare Programming Language (SPL) and loaded from punched cards. The cards are produced by a microcontroller-driven card punch, fed by an AI translation stage that converts a reference implementation written in Rust into SPL.

GNSS, short for Global Navigation Satellite System, is the general term for satellite navigation systems. The United States' GPS is the best known, alongside Europe's Galileo, Russia's GLONASS, and China's BeiDou. Each satellite broadcasts its position and a time signal derived from onboard atomic clocks, so a receiver that hears four or more satellites can solve for both its location and the exact time.

A timing receiver turns that solution into a pulse-per-second (PPS) output aligned to Coordinated Universal Time (UTC), and that pulse is the reference this clock is disciplined to. Using all four systems rather than GPS alone keeps the clock locked when the sky view is poor or one system has an outage.

The project deliberately chains together four eras of computing: 18th-century clockwork, 1960s punched-card media, modern satellite timing, and present-day machine translation of code. Each link in the chain is a real engineering problem, and each hand-off between links is an interface the teams must specify, test, and defend.

The absurdity is the point, but it is load-bearing absurdity. An esoteric target language forces rigorous verification of AI-generated code. A physical card medium forces students to think about encoding, error detection, and data integrity. A pendulum that must track GNSS to the second forces a real control-loop design rather than a software clock that simply resets itself.

The project is intended for a two-semester senior design sequence with a team of roughly 6–9 students drawn from electrical engineering, mechatronics, and computer science.

System architecture

The build path runs one way from Rust source to a punched deck; the only feedback loops are the test gate that rejects bad translations and the clock's own discipline loop.

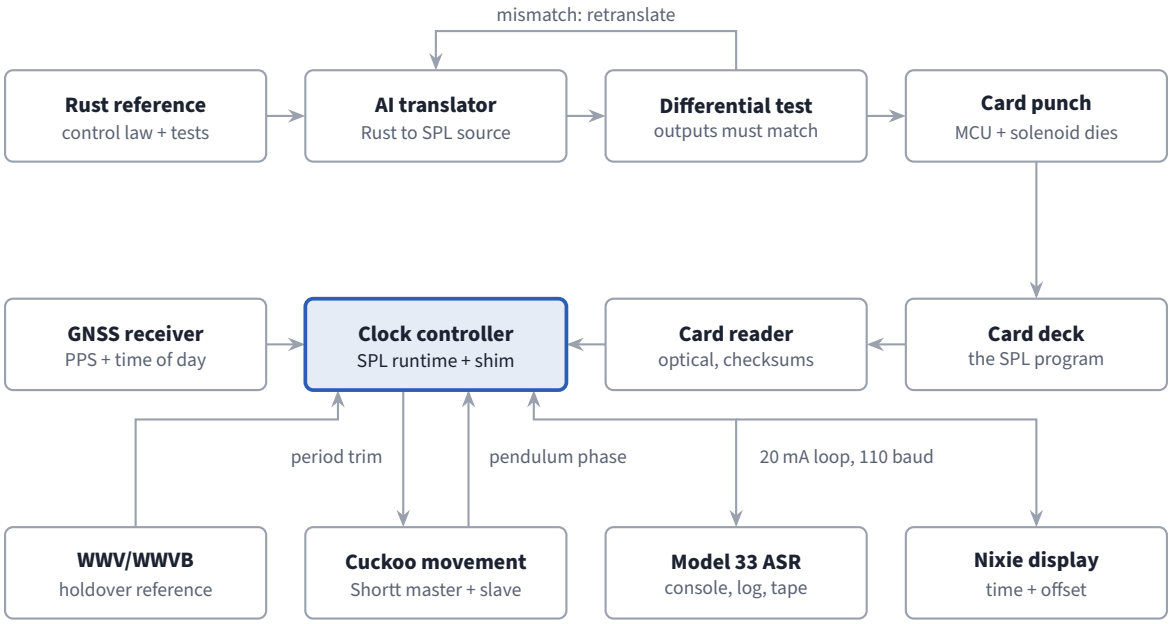


Figure 1. System architecture: build path and clock loop. Rust becomes punched SPL cards that steer a GNSS-locked pendulum.

The controller compares each GNSS pulse-per-second edge with the pendulum's optical gate, and the SPL program turns that phase error into a period trim. The punch-and-read path is offline: cards are punched once per firmware build and read at boot.

Subsystems and team responsibilities

The system divides into six subsystems, each with a lead discipline and at least one supporting discipline so that no team works in isolation.

Subsystem	Lead discipline	Supporting	Core tasks
1. Rust reference firmware	CS	EE	Write the GNSS-discipline algorithm (PPS phase measurement, PI loop, chime scheduling) in Rust; build a test suite and simulator
2. AI Rust-to-SPL translator	CS	—	Prompting/tooling to translate Rust to SPL; automated equivalence testing against the Rust reference
3. Card encoder and punch	Mechatronics	EE, CS	Define a card encoding for SPL source; design and build a microcontroller-driven punch (solenoid or stepper die actuation, card feed, registration)
4. Card reader	EE	Mechatronics	Optical (LED/phototransistor array) or mechanical card reader with checksum validation; streams program into the clock controller
5. Clock controller and SPL runtime	EE	CS	Microcontroller board with GNSS receiver, PPS capture, and an SPL interpreter or spl2c-compiled image with a hardware I/O shim
6. Cuckoo mechanism and discipline actuator	Mechatronics	EE	Shortt-style free master pendulum in a sealed tank, slave pendulum with hit-and-miss synchronizer, optical gate sensing, master rate trim, cuckoo bird and bellows drive, hour/half-hour strike

Table 1. Subsystems, lead and supporting disciplines, and core tasks.

CS students own correctness of the code path from Rust to SPL. Mechatronics students own everything that moves. EE students own timing, sensing, power, and the board-level electronics that tie the other two together.

Model 33 Teletype console

A Model 33 ASR Teletype serves as the clock's operator console, its log printer, and a paper-tape fallback for the card path. Mechatronics restores it, EE builds the interface, and CS writes the console protocol.

- **Console and log:** the controller prints one line per hour (GNSS time, pendulum phase error, trim command) and accepts typed commands such as status or forced holdover. The clock's discipline performance accumulates on a paper roll.
- **Program listings:** before a deck is punched, the SPL source prints on the Teletype for human review, giving every build a hard-copy listing.

- **Paper-tape fallback:** the ASR's tape punch and reader give a second, period-correct program medium if the card punch or reader slips.
- **Interface:** the 33 speaks a 20 mA current loop at 110 baud with two stop bits, not RS-232 levels. EE builds an optoisolated current-loop-to-UART adapter with its own loop supply.

Constraints to plan around: the 33 prints uppercase only, at about 10 characters per second, so a long SPL listing takes a while (tape keeps full ASCII, so case survives on tape). It needs cleaning and lubrication before reliable service. Its character timing comes from a synchronous AC motor, so its baud clock follows mains frequency, not GNSS; timing its start bits against PPS could make a crude grid-frequency monitor as a side experiment.



Figure 2. The card punch and the Teletype console: one makes the program, the other prints its listing and the clock's log. *(Illustration.)*

Additional features

Four additions give each discipline one more visible piece: a second time reference, an audible time announcement, a display, and an optional constellation experiment.

- **WWV/WWVB second reference (EE lead):** a 60 kHz WWVB receiver, or an SDR tuned to WWV on HF, gives the controller an independent reference. During a GNSS outage, holdover is measured against WWV/WWVB instead of free-running, and the two references can be logged side by side. Radio propagation delay limits this to millisecond-class accuracy, which is still far finer than a pendulum needs.
- **Morse time announcement (CS lead):** after the cuckoo calls the hour, the time is sent in CW on a sounder and printed on the Teletype. The Morse encoder is written in Rust and goes through

the same translate-test-punch path, so it ships in SPL like everything else.

- **Nixie display (EE and mechatronics):** GNSS time on Nixie tubes beside the pendulum, with a small readout of the current clock offset so visitors can watch the loop work. The tubes need a high-voltage supply of roughly 170 V DC, which the safety plan must cover.
- **Constellation comparison (EE and CS, optional):** each constellation keeps its own system time: GPS Time, Galileo System Time, BeiDou Time, and GLONASS time tied to UTC(SU). The receiver reconciles them and steers PPS to UTC, but students can also log each system's timing solution separately and compare offsets and stability across the multi-day runs. A dual-frequency timing receiver adds a second experiment: measuring how much ionospheric-delay correction improves timing.

Technical challenges and open questions

The hardest problems sit at the interfaces, not inside the subsystems.

- **SPL has no hardware access.** The language reads and writes characters and integers on standard I/O and nothing else. The runtime must map that stream onto hardware: for example, the SPL program reads PPS-to-pendulum phase error as an integer and writes a trim command as an integer, with a C or Rust shim handling the actual peripherals. Teams must decide how much logic lives in SPL versus the shim, and the grading rubric should require that the control law itself is in SPL.
- **SPL runtime on a microcontroller.** The common spl2c translator emits C that can be cross-compiled, but it is not designed for embedded targets. A small purpose-built SPL interpreter may be easier to fit and to feed from a card reader.
- **Trusting AI-translated code.** LLM translation into an esoteric language will produce plausible but wrong programs. The translator team needs a differential testing harness that runs the Rust reference and the SPL output on identical recorded phase-error sequences and requires matching outputs before anything is punched.
- **Card encoding and program size.** SPL is extremely verbose; a modest control loop may run to thousands of lines. At 80 columns per card, card count and reader reliability become real constraints. Options include a compact token encoding, a per-card sequence number, and per-card checksums.
- **The reader is not optional.** The brief specifies punching cards, but the clock must also read them. That reader is a full subsystem and should be scoped from the start.
- **Disciplining a pendulum.** A mechanical pendulum's period drifts with temperature and amplitude. The discipline loop must estimate phase against GNSS PPS and steer the period through a small actuator, with loop time constants of minutes, not milliseconds. Holdover behavior when GNSS is lost should be specified.
- **Movement: a Shortt–Synchronome-style free pendulum.** The clock uses a Shortt master-and-slave pair instead of an ordinary cuckoo pendulum. A free master pendulum swings in a sealed, partially evacuated tank, receives an impulse every 30 seconds, and does no other work. A slave pendulum drives the cuckoo movement and is kept in step by the classic hit-and-miss synchronizer. GNSS disciplines the master through a very small rate trim, so the system nests two loops: the 1920s mechanical bang-bang loop inside a modern PI loop. Students build a replica, not a restored

original, since surviving Shortt clocks are rare museum pieces. The master's tank, invar rod, and impulse mechanism will be the largest mechatronics task in the project.

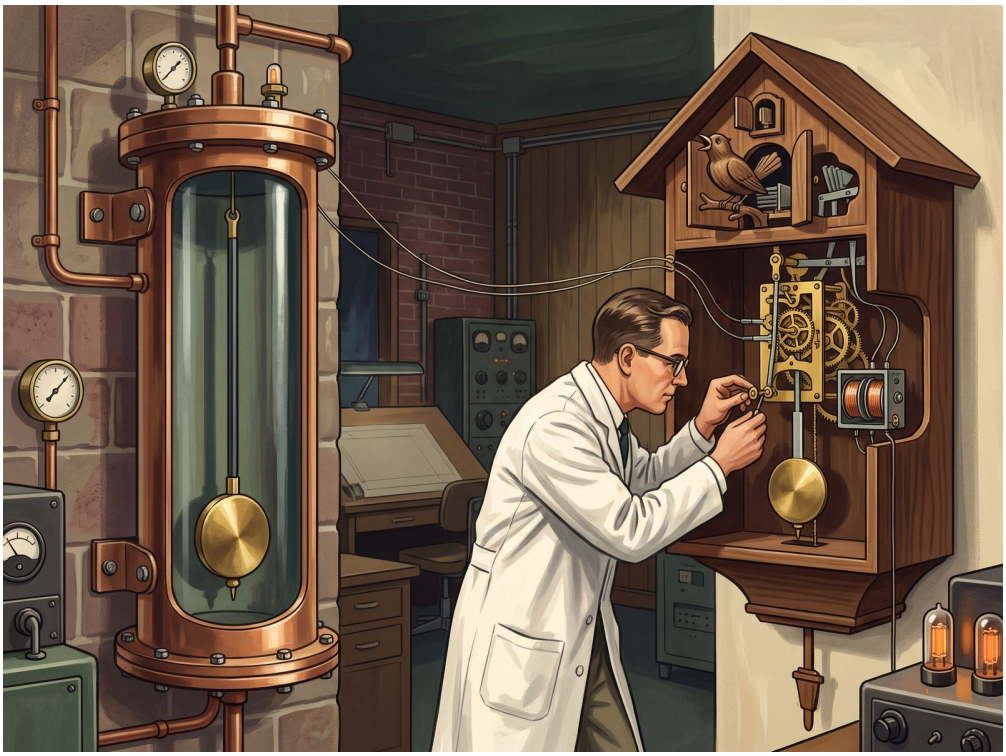


Figure 3. The Shortt arrangement: a free master pendulum in its vacuum tank, and the slave pendulum driving the cuckoo. *(Illustration.)*

Practical requirements

Four requirements must be settled in the first weeks, before any subsystem design is frozen.

- **Budget and sourcing:** a parts list with a fallback for every hard-to-find item. Blank 80-column cards are scarce, so plan to cut cards from card stock of the right thickness. A working Model 33 can take months to find; identify a machine or donor early and budget for restoration parts.
- **Safety:** guards and an interlock on the punch dies, inspection of the Teletype's mains wiring before power-up, and enclosure of the Nixie high-voltage supply. A lockout procedure for each is signed off before students work on them unsupervised.
- **Measurement setup:** a time-interval counter or logic analyzer timestamps the pendulum's optical gate against GNSS PPS. Every timing criterion depends on this instrument, so it must be in hand before integration starts.
- **AI-use policy:** prompts, model versions, and translation outputs are logged for every build, and credit for translated code is documented in the final report. The failed translations, and what the differential tests caught, are a project result in their own right and suitable for a conference poster or paper.

Learning outcomes

By the end of the project, students will be able to:

1. Specify, document, and test interfaces between subsystems built by teams from other disciplines.
2. Design and tune a feedback control loop that steers a physical oscillator to an external time reference, including holdover behavior.
3. Use GNSS timing (PPS, NMEA/UBX messages) and characterize timing error with measured data.
4. Critically evaluate AI-generated code through differential testing, rather than accepting it on inspection.
5. Design a data encoding with error detection for a physical storage medium.
6. Build precision electromechanical actuators and sensors, including solenoid or stepper actuation and optical sensing.
7. Run a staged integration plan with defined fallback paths when a subsystem slips.

These map onto the ABET student outcomes for complex problem solving, design, communication, teamwork, and experimentation.



Figure 4. Integration night: oscilloscope, Teletype log, Nixie display, and the card deck all checked against the sky. *(Illustration.)*

Deliverables, milestones, and evaluation

The full chain must run end to end by mid-spring, leaving the second half of the year for timing measurements and the final demonstration.

The full-chain demo is the critical gate. If the punch or reader slips, teams fall back to a hand-punched short deck or a serial stand-in for the reader so the controller and movement can still be integrated on schedule.

Proposed evaluation criteria, with numeric targets to be set by the faculty sponsor at kickoff:

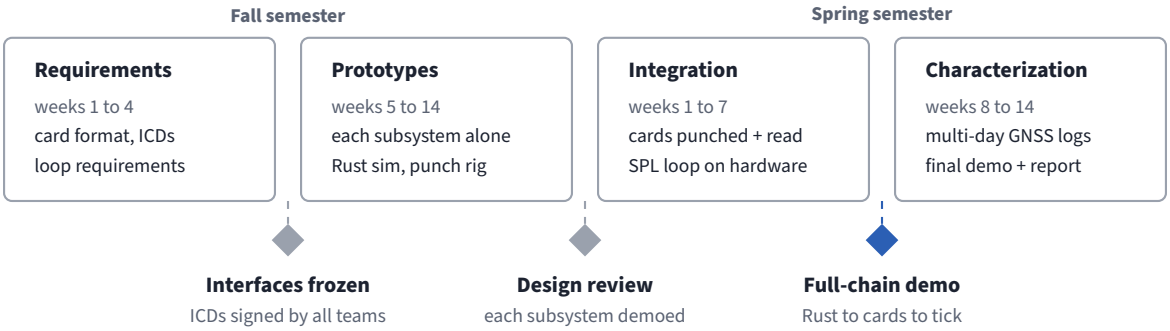


Figure 5. Two-semester schedule: four phases and three gates. The gates keep the chain from slipping into spring.

Deliverable	Owner	Due at
Rust reference firmware and simulator	CS	Design review
AI translator and differential test harness	CS	Design review
Card encoding spec and working punch	Mechatronics	Design review
Card reader with checksum validation	EE	Full-chain demo
Controller board with SPL runtime and I/O shim	EE, CS	Full-chain demo
Disciplined cuckoo movement	Mechatronics, EE	Full-chain demo
Final report, timing data, public demo	All	End of spring

Table 2. Deliverables, owners, and the gate each is due at.

- **Timekeeping:** clock time offset from GNSS over a multi-day run, and drift during a planned GNSS outage.
- **Translation correctness:** SPL output matches the Rust reference on every recorded test vector.
- **Media integrity:** the full deck reads with no uncorrected errors across repeated passes.
- **Function:** the cuckoo calls the correct count on every hour of the test run.
- **Engineering process:** interface documents kept current, design reviews passed, individual contributions documented.

Text drafted with AI assistance (Claude, Anthropic); review by Nathaniel Frissell, W2NAF, is in progress. Illustrations generated with Google Gemini; they are artistic depictions, not drawings of the actual build.

A project of the University of Scranton Amateur Radio Club (W3USR). `https://github.com/w3usr/gnss-cuckoo-clock`. Text licensed CC BY 4.0.